

# On-Edge Mobile Diagnosis of Malaria Parasites using Machine Learning

Meriem Aoudia<sup>1</sup>, Raghad Aldamani<sup>1</sup>, Diao Addeen Abuhani<sup>1</sup>

<sup>1</sup>Department of Computer Science and Engineering, American University of Sharjah, Sharjah, UAE.

Contributing authors: [g00105691@aus.edu](mailto:g00105691@aus.edu); [g00085944@aus.edu](mailto:g00085944@aus.edu); [b00086137@aus.edu](mailto:b00086137@aus.edu);

## Abstract

The abstract Malaria, caused by Plasmodium parasites, remains a significant global health challenge, particularly in resource-constrained settings where access to reliable diagnostic infrastructure is limited. This study explores the integration of machine learning-based neural networks into mobile applications for detecting and classifying Plasmodium trophozoites in blood smear images. Leveraging quantization techniques, including FP16, integer quantization with representative data, and quantization-aware training (QAT), the models were optimized for deployment on edge devices. Evaluation metrics such as accuracy, sensitivity, specificity, mIoU, and Dice Score demonstrated robust performance across architectures like DenseNet121, MobileNetV2, and ResNet50, with MobileNetV2 showing the best balance of efficiency and accuracy under QAT. The developed mobile application further enhances the accessibility and scalability of malaria diagnosis, offering a cost-effective, real-time diagnostic tool for underserved regions.

**Keywords:** Mobile Diagnosis, Malaria, Optimization, On-Edge

## 1 Introduction

Despite the rapid advancements in mobile technology and their growing ubiquity, mobile devices have not yet become integral tools in diagnostic imaging for widespread medical use [1]. This is largely due to concerns over patient privacy [2], the lack of regulatory standards to ensure clinically acceptable image handling and manipulation [3], and doubts regarding whether mobile platforms can genuinely enhance the diagnostic process [4]. Recent studies, however, have started addressing these concerns by highlighting the feasibility of mobile devices for medical diagnostics under appropriate safeguards [5–8]. These findings indicate that the potential for mobile technology in healthcare is both promising and realistic, particularly in resource-limited environments where traditional diagnostic infrastructure is unavailable.

Malaria, a disease caused by Plasmodium parasites and transmitted through the bites of infected Anopheles mosquitoes, remains a major public health challenge in tropical and subtropical regions. The identification of the Plasmodium trophozoite stage in blood smears is a critical diagnostic step, traditionally performed manually under a microscope by trained professionals [9]. However, manual microscopy is time-consuming, requires significant expertise, and may be prone to errors, particularly in high-volume or low-resource settings [10]. Leveraging the computational power of neural networks and the portability of mobile devices offers an innovative pathway to address these challenges.

Deploying neural networks on mobile devices, however, comes with technical hurdles, including the need to optimize models for the limited computational resources and storage capacity of smartphones. For instance, operating systems impose application size restrictions—200 MB for Android’s Google Play [11] and 500 MB for Apple’s App Store [12]. These constraints make it necessary to reduce the size of machine learning models while preserving their diagnostic accuracy. Model quantization,

a technique that reduces model size by converting weight formats (e.g., from float-32 to float-16 or int-8), has proven to be an effective solution [13]. Quantized models significantly lower storage requirements and computational overhead, enabling real-time inference on mobile devices without compromising diagnostic performance.

In this study, we explore the deployment of neural networks on mobile devices for the detection and classification of Plasmodium trophozoite cells in blood smear images. We employ TensorFlow Lite quantization techniques to optimize model size and performance, ensuring seamless functionality on mobile platforms. By integrating machine learning capabilities directly into mobile applications, our approach aims to enhance accessibility, streamline the diagnostic process, and provide a scalable solution for malaria detection in resource-constrained settings. The work presented is available at [GitHub](#).

The remainder of this paper details a background about the topic in general, followed by a related work section highlighting the most recent works in this domain. Then a methodology section detailing our approach. and last but not least, a results and discussion section including the development and quantization of neural network models, the architecture of the mobile application, and a thorough analysis of the trade off between performance and model size.

## 2 Background

### 2.1 Malaria Spread and Infection

Malaria remains a significant global health challenge, with an estimated 249 million cases and 608,000 deaths reported worldwide in 2022. The World Health Organization (WHO) [14] indicates that the African Region disproportionately bears the brunt of this burden, accounting for approximately 94% of malaria cases (233 million) and 95% of malaria deaths (580,000). Notably, children under five years old are particularly vulnerable, constituting about 80% of malaria-related deaths in Africa. This high mortality rate among young children underscores the critical need for effective prevention and treatment strategies in the region.

The standard method for diagnosing malaria involves the microscopic examination of stained blood smears, a technique that has been the cornerstone of malaria diagnosis for over a century [15]. This process entails preparing both thick and thin blood smears, which are then stained—commonly with Giemsa stain—to visualize the parasites under a microscope. The thick smear is more sensitive for detecting the presence of parasites, as it examines a larger volume of blood, while the thin smear assists in identifying the specific species of Plasmodium and determining the level of parasitemia [16]. However, the accuracy of this method heavily depends on the quality of the reagents, the microscope, the blood smear preparation, and the expertise of the laboratory personnel. Studies have shown that while microscopy remains the gold standard for malaria diagnosis, its effectiveness can be compromised by these variables, highlighting the need for well-trained personnel and quality-controlled laboratory environments to ensure reliable results [16].

### 2.2 Mobile Diagnosis

Mobile diagnosis using microscopic images has revolutionized the field of biomedical imaging by enabling compact, cost-effective, and portable solutions [17]. These systems leverage high-resolution smartphone cameras and attachable microscope lenses to capture and analyze microscopic samples, such as blood cells, bacteria, or tissue sections [18]. This approach has been instrumental in diagnosing a wide range of conditions, including bacterial infections [19], hematological disorders [20], and even cancerous cells [21]. By integrating advanced image processing and machine learning algorithms, these platforms can automate complex tasks, such as detecting abnormalities, classifying cell types, or quantifying key metrics, offering reliable results comparable to traditional laboratory methods. The portability of these devices makes them particularly useful in point-of-care diagnostics, remote healthcare settings, and field research, reducing dependence on costly infrastructure and specialized expertise. However, despite their promise, mobile diagnosis systems face several challenges that hinder widespread adoption. Variability in sample preparation, lighting conditions, and image quality can affect the accuracy and reliability of results, requiring robust preprocessing and standardization techniques in addition to the mobile phones own restrictions in terms of size and inference time [18].

## 2.3 Quantization

Quantization is a crucial technique in optimizing neural networks for deployment on resource-constrained devices such as mobile phones and edge computing platforms. It reduces computational cost and memory requirements by manipulating how weights and activations are stored, often representing them with fewer bits [22]. For instance, Floating Point 16 (FP16) quantization involves replacing the standard 32-bit floating-point representation with a 16-bit format, effectively halving memory usage and improving computational efficiency. This method accelerates computation while having minimal impact on overall model performance, making it suitable for edge devices such as smartphones [22]. FP16 quantization is particularly valuable when short inference times are critical, as it allows real-time performance without requiring additional hardware modifications. These benefits make FP16 quantization a widely used approach in the deployment of lightweight neural networks.

More advanced quantization techniques address the potential accuracy loss associated with reduced precision. Integer quantization, for example, uses representative data—a subset of real samples—to simulate model inference during the quantization process. By capturing the distribution of weights and activations, this approach ensures that the quantized model accurately reflects the behavior of the original model on unseen data [23]. Quantization-aware training (QAT) goes a step further by incorporating the quantization process during the training phase, applying quantization functions to both activations and weights as the model learns. This allows the neural network to adapt to the reduced precision environment, resulting in higher accuracy when deployed on low-bit platforms such as mobile devices or specialized hardware [24]. Together, these methods—ranging from FP16 quantization to QAT—make quantization an indispensable tool for deploying efficient and accurate neural networks in environments with resource constraints.

## 3 Related Works

The application of machine learning for malaria parasite diagnosis using microscopic images has been somewhat explored in recent years. Works in this domain can be categorized based on the general objective whether it is direct classification or further cells localization and detection. In this section we provide a critical review of the state-of-the-art advances in malaria parasite applications using machine learning with respect to the trade off between performance and model size.

Distinguishing malaria infected blood smears from benign ones is a more common problem to address than localization overall. This can be explained with the fact that the data availability for classification is higher. The majority of works rely on well-known datasets including NIH dataset [25], MP-IDB [26], MP-IDB2 [27], IML\_Malaria [28], Malaria-Detection-2019 [29], and most recently Lacuna Malaria Detection Challenge dataset [30]. These datasets collectively encompass thousands of annotated images of red blood cells, detailing various species and life-cycle stages of Plasmodium parasites. They serve as valuable resources for training and evaluating machine learning models aimed at automating malaria diagnosis, thereby enhancing the accuracy and efficiency of detecting and classifying malaria infections.

The application of deep learning techniques to malaria detection from blood smear images has seen significant advancements, particularly in classification and object detection tasks. Several studies have developed models to classify blood smear images as infected or uninfected with malaria parasites. For instance, a study used a ResNet-based CNN model for the classification of microscopic blood smear images as either malaria-infected or uninfected, with promising results for automated detection [31]. A custom CNN trained on 1,819 thick blood smear images effectively distinguished between parasite-infected cells and the background, enabling smartphone-based detection [25]. Others focused on enhancing classification accuracy by fine-tuning deep learning models. For example, a model using pre-trained networks such as VGG16 and DenseNet was applied to classify malaria parasites in thin blood smear images, showing the effectiveness of fine-tuned CNNs in improving classification performance [28].

A significant number of studies also employed object detection methods for malaria diagnosis, particularly with the aim of detecting small and challenging objects such as parasites in thick blood smears. Several models, such as Faster R-CNN, RetinaNet, and YOLOv5, were tested on different datasets with varying success. For example, a customized Faster-RCNN model was used for small object detection, focusing on parasite identification from 2,967 images of thick blood smears, providing insights into the future potential for reducing false positives [32]. Additionally, a system using

**Table 1** Summary of Datasets for Malaria Diagnosis

| Dataset Name                  | Description                                                                                                | Purpose        |
|-------------------------------|------------------------------------------------------------------------------------------------------------|----------------|
| NIH Dataset [25]              | A well-known dataset for malaria diagnosis, featuring annotated images for classification.                 | Classification |
| MP-IDB [26]                   | Contains annotated images of red blood cells, detailing Plasmodium parasite species and life-cycle stages. | Localization   |
| MP-IDB2 [27]                  | An extended version of MP-IDB with additional data for enhanced robustness.                                | Localization   |
| IML Malaria [28]              | Features annotated red blood cell images with malaria-infected and non-infected samples.                   | Classification |
| Malaria Detection 2019 [29]   | Includes images with ground truth labels and life-stage annotations for <i>Plasmodium</i> parasites.       | Localization   |
| Lacuna Malaria Detection [30] | Provides annotated blood smear images for automated malaria parasite detection.                            | Localization   |

YOLOv5 demonstrated high efficacy for object detection in multi-class scenarios, categorizing cells as infected or uninfected [33]. These detection-based models typically show faster inference times and improved accuracy, as evidenced by their implementation on smartphone platforms for real-time diagnosis [34]. In recent works, other innovative detection approaches have been explored. For example, YOLOv5 was employed for detecting thin-blood-smear images from the SmartMalariaNET dataset, achieving an average precision of 90.8%, which provided strong performance in malaria cell detection. However, the study did not mention how the model could scale across different malaria species [35]. In another study, Mask R-CNN was utilized on 297 microscopy images, achieving a detection accuracy of 94.57%, with the focus on automating diagnosis and using mAP as the main evaluation metric [36]. Additionally, a more recent study utilized a variety of object detection models such as YOLOv8, RetinaNet, and Faster R-CNN, combining them using different ensembling strategies to significantly improve detection accuracy. YOLOv8 achieved an mAP of 0.9031, and the ensemble model reached an mAP of 0.9186, highlighting the benefit of combining multiple detection techniques to overcome individual model limitations [37]. The application of transfer learning techniques plays a crucial role in improving the performance of malaria detection systems. Several studies have adopted pre-trained models to detect and classify parasites, reducing the need for large annotated datasets. For instance, a study evaluated various deep learning models, including Faster-RCNN, SSD MobileNet V2, and RetinaNet, on a dataset of 643 images from a referral hospital, demonstrating that pre-trained networks could be adapted for high-quality parasite detection [38]. Furthermore, combining transfer learning with additional techniques such as random forest classifiers has been shown to enhance detection capabilities, as demonstrated in the DLRFNet framework, which achieved high accuracy in classifying malaria parasite images [39]. Smartphone-based applications for real-time malaria detection are increasingly viable, particularly in resource-limited settings. Some research has developed mobile-friendly deep learning algorithms that allow for immediate screening on portable devices. For instance, the Malaria Screener app employs pre-trained CNN models to detect malaria in blood smears, with a focus on cost-effectiveness [40]. These innovations are improving access to effective malaria detection tools, especially in remote areas.

## 4 Methodology

The methodology followed in this work is shown in Figure 1. The process began with dataset pre-processing and augmentation, followed by the training of multiple models to classify the data into three distinct classes. These models were then quantized using various techniques before they were evaluated and analyzed based on their performance. Finally, the best-performing model was selected and deployed onto a mobile application for real-time malaria detection.

### 4.1 Dataset Preprocessing

The dataset employed in this study was the publicly available Lacuna Malaria Detection dataset containing 2,747 labeled images of blood smear samples. Each image was annotated with bounding boxes and the respective class, identifying three instances: Negative samples, White Blood Cells

**Table 2** Summary of Related Papers on Malaria Detection for Mobile Diagnosis.

| Work | Model(s) Used                                            | Quantized | Deployed | Purpose        |
|------|----------------------------------------------------------|-----------|----------|----------------|
| [32] | Customized Faster-RCNN                                   | ×         | ×        | Detection      |
| [31] | ResNet                                                   | ×         | ×        | Classification |
| [41] | Deep Transfer Graph Convolutional Network (DTGCN)        | ×         | ×        | Classification |
| [39] | Deep-CNN-RF architectures                                | ×         | ×        | Classification |
| [42] | Multiple CNN models                                      | ×         | ×        | Detection      |
| [43] | Customized YOLOv5                                        | ×         | ×        | Detection      |
| [44] | YOLOv4-based models                                      | ×         | ×        | Detection      |
| [45] | Faster-RCNN with ResNet-50                               | ×         | ×        | Detection      |
| [46] | GoogLeNet, SVM, Ensemble methods                         | ×         | ×        | Classification |
| [36] | Mask R-CNN                                               | ×         | ×        | Detection      |
| [37] | YOLOv8, RetinaNet, Faster R-CNN, EfficientDet, CenterNet | ×         | ×        | Detection      |
| [47] | Faster R-CNN, YOLO                                       | ×         | ×        | Classification |
| [48] | YOLO-SPAM                                                | ×         | ×        | Detection      |
| [25] | Custom CNN                                               | ×         | ✓        | Detection      |
| [49] | PVF-Net                                                  | ×         | ✓        | Classification |
| [38] | Faster R-CNN, SSD MobileNet V2, RetinaNet                | ×         | ✓        | Detection      |
| [28] | U-Net, VGG16, ResNet50v2, DenseNet                       | ×         | ✓        | Detection      |
| [33] | YOLOv5, Faster R-CNN, RetinaNet, SSD                     | ×         | ✓        | Detection      |
| [34] | Faster R-CNN, SSD MobileNet V2                           | ✓         | ✓        | Detection      |
| [40] | Pre-trained CNN models                                   | ✓         | ✓        | Classification |

(WBC) and Trophozoites cells. In order to make the data suitable for classification, the presence of at least one Trophozoite in an image was considered a Malaria Positive case. Images labeled exclusively as WBC were categorized as Suspicious, while any image with no labels was classified as Negative. The dataset initially included some class imbalance, particularly for the Suspicious class, which had only 41 images. To address this, we applied various data augmentation techniques such as flipping, scaling, and rotation to generate more samples for the Suspicious class and balance the dataset, ensuring better model performance and generalization. The final dataset used included 2018 Positive samples, 688 Negative samples, and 500 suspicious samples with a 70-10-20 train-val-test split for validation and testing purposes.

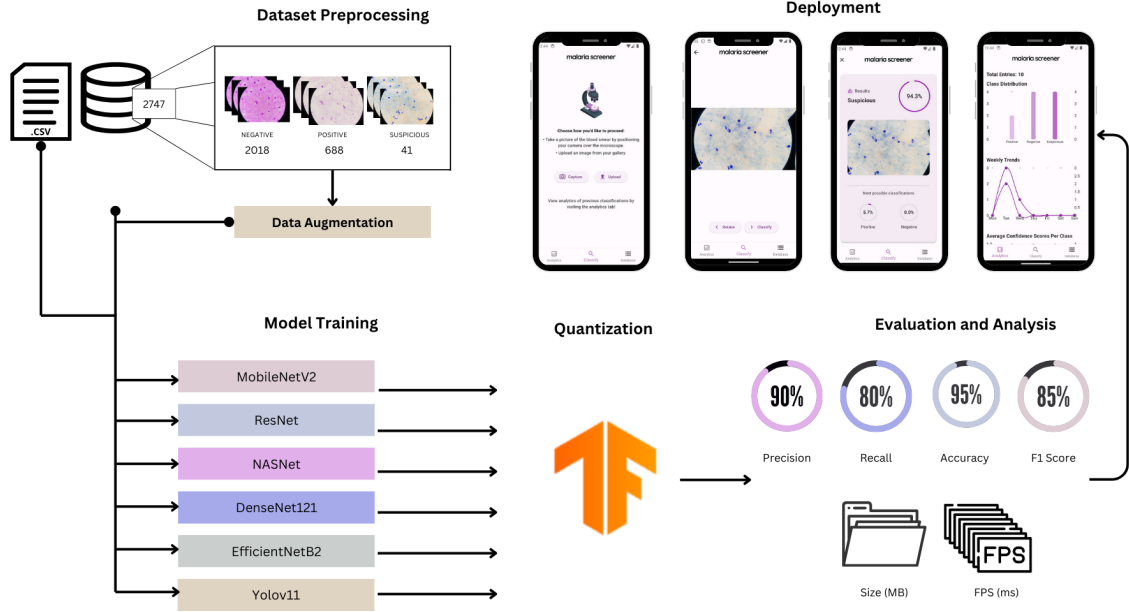
## 4.2 Transfer Learning

Transfer learning [50] is a pivotal technique in deep learning that enhances training efficiency by leveraging the knowledge gained from a pre-trained model. Instead of training a neural network from scratch, transfer learning uses a model pre-trained on a large dataset as a starting point, allowing it to adapt to new tasks with less computational effort and training data. This approach accelerates convergence, improves performance, and enables the network to focus on adapting to the specific nuances of the new dataset rather than relearning basic feature extraction.

In this study, five pre-trained neural network architectures trained on the ImageNet dataset [51] were evaluated for their suitability in resource-constrained environments. The selected models were chosen based on their compact size—approximately 100MB or less—ensuring compatibility with edge devices such as mobile phones. A detailed description of each architecture is provided below.

### 4.2.1 ResNet50

ResNet50 [52] is a deep convolutional neural network with 50 layers, featuring residual connections that address the vanishing gradient problem and enable efficient training of deeper networks. These connections allow gradients to flow more effectively, resulting in improved accuracy and faster convergence compared to traditional architectures. Pre-trained on ImageNet, ResNet50 has demonstrated



**Fig. 1** Overall view of the methodology followed in this work. The data was first processed and augmented. Later, different models were trained to classify the data into three classes. Then, the models were quantized using different techniques before being evaluated and analyzed. Finally, the best performing model was deployed on a mobile phone.

strong performance across various classification tasks and serves as a robust baseline for fine-tuning on new datasets.

#### 4.2.2 MobileNetV2

MobileNetV2 [53] is a lightweight neural network designed specifically for mobile and embedded applications. It employs depthwise separable convolutions and inverted residual blocks to achieve a balance between computational efficiency and accuracy. With its minimal resource requirements, MobileNetV2 is well-suited for real-time inference on devices with limited processing power, making it a strong candidate for tasks involving mobile deployment.

#### 4.2.3 DenseNet121

DenseNet121 [54] introduces dense connectivity, where each layer is directly connected to all subsequent layers. This architecture encourages feature reuse, improves gradient flow, and reduces the risk of overfitting while maintaining computational efficiency. DenseNet121 achieves high performance in computer vision tasks with fewer parameters compared to traditional networks, making it a compelling choice for deployment on resource-constrained devices.

#### 4.2.4 NASNetMobile

NASNetMobile [55] is a compact version of the NASNet architecture, developed using neural architecture search (NAS) to optimize performance for specific tasks. By employing reinforcement learning or evolutionary algorithms, NASNet explores vast design spaces to identify efficient and accurate configurations. NASNetMobile is tailored for mobile applications, featuring fewer parameters and computational demands compared to larger NASNet variants, making it ideal for lightweight deployment scenarios.

#### 4.2.5 EfficientNetB2

EfficientNetB2 [56] belongs to the EfficientNet family, known for its compound scaling approach, which balances depth, width, and resolution for optimal efficiency. By leveraging this scaling method, EfficientNetB2 achieves state-of-the-art performance with minimal resource consumption, making it highly effective for applications requiring both accuracy and efficiency. Its lightweight design and advanced building blocks ensure fast inference and training while maintaining model quality.



#### 4.2.6 YOLOv11

YOLOv11 [57] is an evolution of the You Only Look Once (YOLO) object detection framework, is designed for high-speed and accurate real-time object detection tasks. Building on the advancements of its predecessors, YOLOv11 introduces optimized feature extraction layers and a streamlined architecture to enhance both detection precision and inference speed. Its ability to detect multiple objects in a single pass makes it ideal for real-world applications requiring rapid decision-making, such as autonomous vehicles, surveillance systems, and medical diagnostics. YOLOv11 further incorporates efficient post-processing techniques to minimize computational overhead, ensuring deployment feasibility on resource-constrained devices without sacrificing model performance.

### 4.3 Optimization and Quantization

Model quantization [22] is a powerful optimization technique designed to reduce the computational complexity and memory requirements of neural networks, making them more suitable for deployment on resource-constrained devices such as mobile phones, IoT devices, and edge hardware. Quantization achieves this by reducing the bit precision used to represent model weights and activations, thereby decreasing model size, improving inference speed, and minimizing energy consumption. While quantization typically introduces some loss in model performance, the extent of this degradation largely depends on the quantization method employed and the strategies used to mitigate its impact. In this study, we utilized TensorFlow Lite to implement three quantization approaches: FP16 floating-point quantization, integer quantization with representative data, and quantization-aware training (QAT). These techniques were chosen for their ability to balance trade-offs between model size, performance, and inference efficiency.

#### 4.3.1 Floating-Point Quantization

Floating-point 16 (FP16) quantization converts the model’s weights and activations from the default 32-bit floating-point (FP32) representation to a 16-bit floating-point format. This transition effectively halves the memory footprint of the model while maintaining a high level of precision, ensuring minimal impact on overall accuracy. The reduced memory and computational requirements also result in faster inference times, making FP16 quantization particularly advantageous for deployment on edge devices, such as smartphones and embedded systems. By enabling compatibility with modern hardware accelerators, FP16 quantization strikes an optimal balance between efficiency and performance.

#### 4.3.2 Integer Quantization with Representative Data

Integer quantization is a more aggressive technique that replaces floating-point representations with integer values, dramatically reducing the memory footprint and computational requirements of the model. To mitigate the performance degradation commonly associated with this process, a representative dataset is employed during quantization [23]. This subset of real data samples is carefully selected to reflect the distribution of the training data, ensuring that the quantized model can generalize well to unseen data. The use of representative data helps align the quantized activations and weights with the original model’s behavior, preserving accuracy while enabling significant reductions in model size and latency. In this study, we leveraged techniques from representation learning to ensure that the chosen representative samples were unbiased and encapsulated the diversity of the full dataset, minimizing the influence of data imbalance.

#### 4.3.3 Quantization-Aware Training (QAT)

Quantization-aware training (QAT) [24] is an advanced technique that incorporates the effects of quantization during the training process, enabling the model to adapt to the constraints of reduced precision. By simulating the quantization of weights and activations during forward and backward passes, QAT allows the model to learn representations that are inherently robust to quantization noise. This results in a model that maintains higher accuracy when deployed on low-bit platforms, such as 8-bit integer hardware. QAT is particularly valuable for scenarios where the highest levels of efficiency are required without sacrificing critical performance metrics. In this study, QAT was implemented to fine-tune the models, ensuring that the benefits of quantization could be realized without compromising their ability to deliver reliable predictions in real-world applications.

## 4.4 Evaluation Metrics

To comprehensively evaluate the performance of machine learning models in image classification, several metrics are utilized. These metrics assess various aspects of model performance, including accuracy, sensitivity, specificity, and computational efficiency. The metrics employed in this study are outlined below.

### 4.4.1 Accuracy

Accuracy is a fundamental metric that measures the overall correctness of a model's predictions. It is defined as the ratio of correctly predicted outcomes, both positive (true positives, TP) and negative (true negatives, TN), to the total number of predictions.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

### 4.4.2 Specificity

Specificity, also known as the true negative rate, evaluates the model's ability to correctly identify negative cases. It is calculated by dividing the number of true negative predictions (TN) by the sum of true negatives and false positives (FP).

$$Specificity = \frac{TN}{TN + FP} \quad (2)$$

### 4.4.3 Sensitivity

Sensitivity, or recall, measures the model's effectiveness in correctly identifying positive cases. It is determined by dividing the number of true positive predictions (TP) by the total of true positives and false negatives (FN).

$$Sensitivity = \frac{TP}{TP + FN} \quad (3)$$

### 4.4.4 F1-Score

The F1-score provides a harmonic mean of sensitivity and precision, offering a balanced evaluation of a model's performance, particularly in scenarios where the distribution of classes is imbalanced. Precision is defined as the ratio of true positives (TP) to the sum of true positives and false positives (FP).

$$F1 = \frac{2 \times Specificity \times Sensitivity}{Specificity + Sensitivity} \quad (4)$$

### 4.4.5 Mean Intersection over Union (mIoU)

Mean Intersection over Union (mIoU) is a standard evaluation metric used in segmentation tasks to measure the overlap between predicted and ground truth masks. It calculates the average IoU across all classes by dividing the intersection of the predicted and ground truth areas by their union.

$$mIoU = \frac{1}{n} \sum_{i=1}^n \frac{|P_i \cap G_i|}{|P_i \cup G_i|} \quad (5)$$

where  $P_i$  represents the predicted mask for class  $i$ ,  $G_i$  represents the ground truth mask,  $\cap$  denotes the intersection,  $\cup$  denotes the union, and  $n$  is the total number of classes.

### 4.4.6 Dice Score

Dice Score, also known as the Sørensen-Dice coefficient, evaluates the similarity between two sets, typically the predicted and ground truth masks. It is particularly sensitive to small regions and is



widely used in medical image analysis. Dice Score is calculated as twice the intersection of predicted and ground truth areas divided by their combined area.

$$Dice\ Score = \frac{2|P \cap G|}{|P| + |G|} \quad (6)$$

where  $P$  is the predicted set,  $G$  is the ground truth set, and  $\cap$  denotes their intersection. A Dice Score of 1 indicates perfect overlap, while 0 indicates no overlap.

#### 4.4.7 Inference Time

Inference time quantifies the duration required for the model to process an input and produce an output. It is a critical metric for assessing the real-time performance and operational efficiency of machine learning models, particularly in applications where quick response times are essential for user experience.

## 5 Results and Discussion

This section presents the achieved results in terms of models performance, size optimization, inference time, and on edge deployment followed by a thorough discussion highlighting the key findings in each subsection.

### 5.1 Models Performance

**Table 3** Performance Metrics of CNN Architectures with Quantization Techniques

| Model          | Quantization | Accuracy | F1-Score | Sensitivity | Specificity |
|----------------|--------------|----------|----------|-------------|-------------|
| DenseNet121    | Actual       | 0.9891   | 0.9892   | 0.9891      | 0.9894      |
|                | FP16         | 0.9870   | 0.9880   | 0.9881      | 0.9799      |
|                | INT          | 0.5398   | 0.5681   | 0.5398      | 0.8390      |
|                | QAT          | 0.9875   | 0.9876   | 0.9875      | 0.9878      |
| EfficientNetB2 | Actual       | 0.9548   | 0.9540   | 0.9548      | 0.9538      |
|                | FP16         | 0.9522   | 0.9520   | 0.9518      | 0.9518      |
|                | INT          | 0.9485   | 0.9478   | 0.9485      | 0.9509      |
|                | QAT          | 0.9406   | 0.9408   | 0.9406      | 0.9412      |
| MobileNetV2    | Actual       | 0.9891   | 0.9890   | 0.9891      | 0.9890      |
|                | FP16         | 0.9871   | 0.9710   | 0.9890      | 0.9883      |
|                | INT          | 0.9797   | 0.9794   | 0.9790      | 0.9796      |
|                | QAT          | 0.9906   | 0.9906   | 0.9906      | 0.9906      |
| NASNetMobile   | Actual       | 0.9641   | 0.9648   | 0.9641      | 0.9670      |
|                | FP16         | 0.9632   | 0.9629   | 0.9621      | 0.9630      |
|                | INT          | 0.4711   | 0.5040   | 0.4711      | 0.5994      |
|                | QAT          | 0.9672   | 0.9665   | 0.9672      | 0.9670      |
| ResNet50       | Actual       | 0.9922   | 0.9923   | 0.9922      | 0.9926      |
|                | FP16         | 0.9832   | 0.9823   | 0.9922      | 0.9821      |
|                | INT          | 0.9938   | 0.9938   | 0.9938      | 0.9939      |
|                | QAT          | 0.9906   | 0.9906   | 0.9906      | 0.9906      |

Generally speaking, all models performed relatively well achieving accuracy performance ranging between 99.22% for ResNet50 and 95.48% for EfficientNetB2 As can be seen in Table 3. For DenseNet121, the results highlight a strong balance between accuracy, F1-score, sensitivity, and specificity under FP16 and QAT, with only minimal performance degradation compared to the original model. However, INT quantization caused a significant drop in accuracy and F1-score (53.98%

and 56.81%, respectively), indicating a trade-off between speed and precision. Similarly, EfficientNetB2 demonstrated stable performance with FP16 and INT, but a noticeable decline in accuracy and sensitivity was observed with QAT, suggesting that this method may not be as effective for this architecture. MobileNetV2 consistently showed high accuracy and F1-scores, with QAT achieving the best performance metrics (99.06% for both accuracy and F1-score), underscoring its suitability for real-time and resource-constrained environments. For NASNetMobile, FP16 and QAT maintained metrics close to the original model, but INT resulted in a dramatic performance drop, with accuracy and sensitivity plummeting to 47.11%. This highlights the challenges of using INT quantization with more complex architectures. ResNet50 demonstrated exceptional robustness, achieving the highest accuracy (99.22%) and F1-score with INT, making it well-suited for applications where precision is paramount. However, FP16 resulted in a notable reduction in specificity, which may limit its utility in certain use cases. Overall, the table demonstrates that while quantization techniques such as FP16 and QAT are effective for maintaining performance while reducing computational demands, their efficacy varies significantly depending on the model architecture. These findings underscore the importance of tailoring quantization strategies to the specific CNN architecture to optimize both accuracy and efficiency. In terms of object detection, Yolov11 model achieved an mIoU score around 74.28% and a Dice score of around 72.29%.

## 5.2 Model Size

| Model              | DenseNet      | EffNetB2      | MobileNet     | NASNet        | ResNet        |
|--------------------|---------------|---------------|---------------|---------------|---------------|
| <b>Original</b>    | 33.7          | 38.5          | 16.5          | 24.2          | 102.4         |
| <b>FP16</b>        | 14.4          | 16.1          | 5.5           | 9.3           | 46.8          |
| <b>INT-REP</b>     | 7.5           | 9.4           | 3.2           | 5.6           | 24.1          |
| <b>QAT</b>         | 14.2          | 16.0          | 5.4           | 9.2           | 46.6          |
| <b>% Max. Opt.</b> | <b>77.74%</b> | <b>75.58%</b> | <b>80.61%</b> | <b>76.86%</b> | <b>76.46%</b> |

**Table 4** Quantization Technique Impact on Model Size (MB)

The quantization techniques demonstrate significant reductions in model size, offering promising avenues for deploying deep learning models on resource-constrained devices. The INT-REP quantization method achieved the highest compression across all models, with MobileNetV2 experiencing the most substantial reduction, shrinking from 16.5 MB to 3.2 MB, representing an 80.61% optimization. Similarly, DenseNet121 and EfficientNetB2 were compressed by approximately 77.74% and 75.58%, respectively. This substantial reduction in size highlights INT-REP’s effectiveness in optimizing storage without substantial computational trade-offs. FP16 and QAT quantizations also resulted in significant size reductions, with QAT retaining a comparable size to FP16 but with enhanced performance metrics, as highlighted in earlier sections.

When correlated with performance results, these size reductions emphasize the trade-offs inherent in quantization. For example, INT-REP achieved the smallest model sizes but at the cost of reduced accuracy and sensitivity in several models, particularly DenseNet121 and NASNetMobile. On the other hand, FP16 and QAT balanced size reduction with stable performance, as seen in MobileNetV2 and ResNet50, which maintained high accuracy and F1-scores despite the compression. These findings underscore the importance of aligning model size optimization with performance requirements, particularly in real-time applications or edge deployments where both storage efficiency and predictive accuracy are critical.

## 5.3 Inference Time and FPS

The table illustrates the impact of various quantization techniques—FP16, INT-REP, and QAT—on frames per second (FPS) for five CNN models: DenseNet121, EfficientNetB2, MobileNetV2, NASNetMobile, and ResNet50. FPS is a critical metric that reflects the efficiency of models in processing inputs in real-time applications. MobileNetV2 consistently demonstrates the highest FPS across all quantization techniques, with QAT achieving an exceptional 15.65 FPS, making it highly suitable

**Table 5** Quantization Technique Impact on FPS (s)

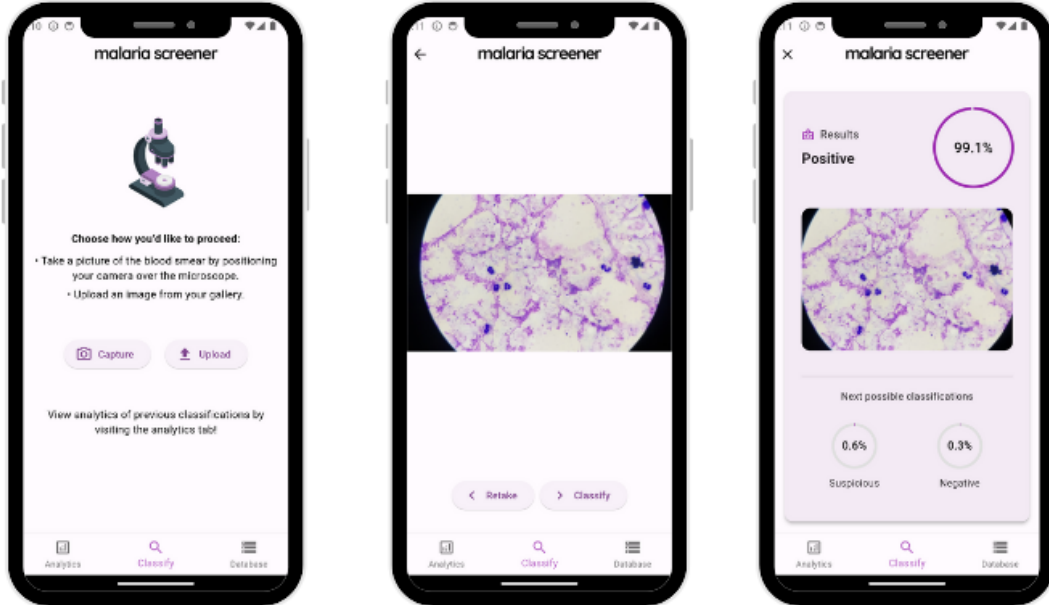
| Model           | DenseNet | EffNetB2 | MobileNet | NASNet | ResNet |
|-----------------|----------|----------|-----------|--------|--------|
| <b>Original</b> | 1.24     | 1.91     | 3.88      | 2.44   | 1.16   |
| <b>FP16</b>     | 1.28     | 1.99     | 10.41     | 3.92   | 0.85   |
| <b>INT-REP</b>  | 1.85     | 3.87     | 9.81      | 1.50   | 1.45   |
| <b>QAT</b>      | 2.39     | 2.84     | 15.65     | 6.15   | 1.17   |

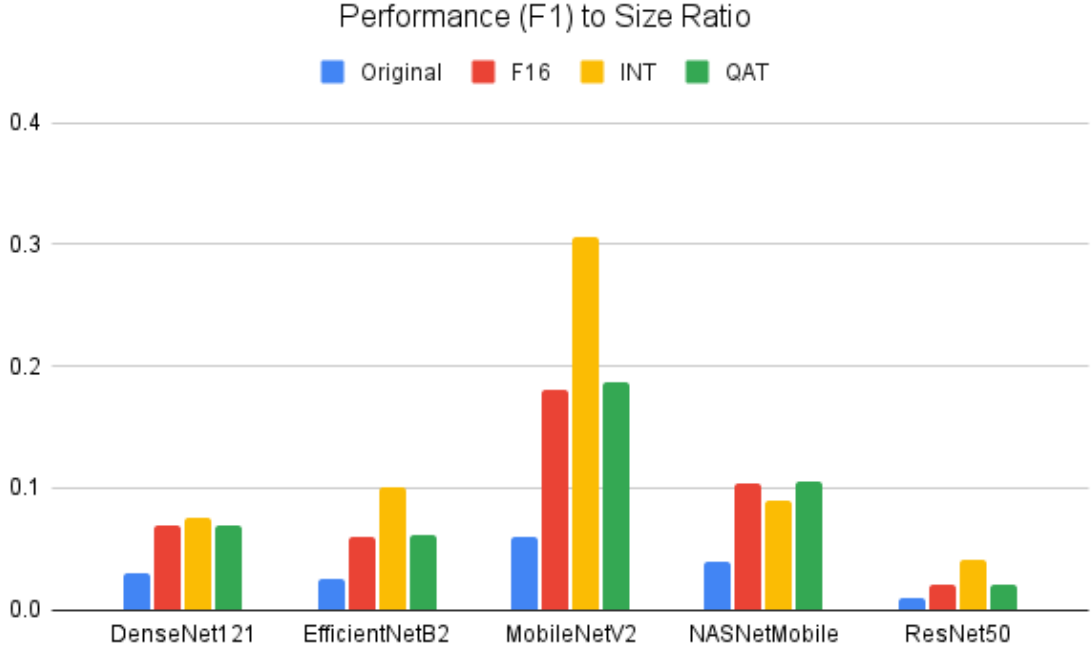
for latency-sensitive scenarios. Similarly, QAT improves DenseNet121’s FPS to 2.39, nearly doubling the baseline of 1.24, while NASNetMobile also benefits significantly, achieving 6.15 FPS compared to its original 2.44 FPS.

INT-REP, known for aggressive compression, shows mixed results: it improves FPS for EfficientNetB2 (3.87 FPS) but slightly reduces it for NASNetMobile (1.50 FPS). FP16 provides a moderate boost in FPS across most models, particularly for MobileNetV2, which increases from 3.88 FPS (Original) to 10.41 FPS. ResNet50, however, experiences inconsistent gains, with FP16 reducing FPS to 0.85, indicating that this technique might not always optimize larger, more complex models. Overall, QAT delivers the best balance between FPS improvements and maintaining performance consistency, making it the most versatile approach for optimizing models for real-time deployments.

## 5.4 Mobile Application Deployment

To deploy the classification models on edge-devices, specifically smartphones, we designed an Android-based mobile application that is easy to use and deployment ready. The app was developed using Flutter framework on Android Studio with a backend connection to Cloud Firestore. The decision to use the Flutter framework was driven by its extensive library of packages, which effectively handle various aspects of model deployment. Additionally, Cloud Firestore provided a simple yet secure method for storing and accessing data throughout the application. The application itself is designed with multiple functional screens. It guides users through the classification and detection process seamlessly, as illustrated in Figure 2. It also features a dedicated screen for viewing previous classification records and an analytics screen that provides an overview of the classifications conducted. The app’s primary purpose is to classify images taken using smartphone cameras mounted

**Fig. 2** Overview of classification process through the mobile application.



**Fig. 3** Performance (F1) to Size Ratio across different models.

on microscopes. Once an image is either captured or uploaded from the user’s gallery, the app invokes the models for instant classification and detection. For the classification task, MobileNetV2 quantized using QAT was chosen due to its high accuracy yet small size as compared to others as shown in Figure 3, as well as YOLOv11 for the detection task. Both models were converted to TensorFlow Lite format, as optimal model size and inference time are critical for resource-limited devices. However, the conversion to TensorFlow Lite introduces challenges as reducing the model size can lead to performance degradation. These challenges are particularly significant due to the platform-imposed size limitations on mobile applications. For instance, Google Play enforces a maximum application size of 200 MB [58], while the App Store allows for a slightly higher limit of 500 MB [59]. These constraints further emphasize the importance of quantization methods, which minimize the trade-off between model size and performance by preserving accuracy and efficiency even within size constraints. By leveraging quantization methods, the final application size was reduced to 97 MB, well within Android’s Google Play size limit. This success demonstrates the feasibility of integrating advanced classification and detection models into resource-constrained mobile applications, making them accessible and effective for real-world use.

## 6 Conclusion

This work underscores the transformative potential of integrating mobile technology with machine learning to tackle critical challenges in malaria diagnosis, particularly in resource-constrained settings. By leveraging state-of-the-art quantization techniques such as FP16, INT-REP, and QAT, the neural network models were meticulously optimized for deployment on edge devices. These techniques not only reduced model size and inference time significantly but also preserved high levels of diagnostic accuracy, demonstrating that computational efficiency and performance can coexist. Among the architectures evaluated, MobileNetV2 stood out for its lightweight design, high frames per second (FPS) under QAT, and suitability for real-time applications, making it ideal for field use where timely diagnostics are essential.

The mobile application developed in this study exemplifies how such optimized models can be seamlessly integrated into point-of-care settings, providing accessible, cost-effective, and scalable diagnostic solutions to under-served regions. This innovation has the potential to reduce reliance on laboratory infrastructure, empowering healthcare providers in remote and low-resource areas to make informed decisions swiftly. Beyond malaria diagnosis, the approach laid out in this work can

serve as a foundation for developing applications targeting other infectious diseases, broadening the scope of impact on global health. Future research should prioritize enhancing model robustness under diverse environmental conditions, including variability in image quality, lighting, and noise, to ensure reliability in real-world scenarios. Additionally, expanding the framework to incorporate multi-disease detection capabilities and adapting the application for emerging pathogens will be critical in addressing evolving healthcare challenges worldwide.

## References

- [1] Kufel, J., Bargieł, K., Koźlik, M., Czogalik, L., Dudek, P., Jaworski, A., Magiera, M., Bartnikowska, W., Cebula, M., Nawrat, Z., *et al.*: Usability of mobile solutions intended for diagnostic images—a systematic review. In: Healthcare, vol. 10, p. 2040 (2022). MDPI
- [2] Flaherty, J.L.: Digital diagnosis: privacy and the regulation of mobile phone health applications. *Am. J. L. & Med.* **40**, 416 (2014)
- [3] Hirschorn, D.S., Choudhri, A.F., Shih, G., Kim, W.: Use of mobile devices for medical imaging. *Journal of the American College of Radiology* **11**(12), 1277–1285 (2014)
- [4] Venson, J.E., Bevilacqua, F., Berni, J., Onuki, F., Maciel, A.: Diagnostic concordance between mobile interfaces and conventional workstations for emergency imaging assessment. *International journal of medical informatics* **113**, 1–8 (2018)
- [5] Tovino, S.A.: Privacy and security issues with mobile health research applications. *The Journal of Law, Medicine & Ethics* **48**(1\_suppl), 154–158 (2020)
- [6] Sahin, V.H., Oztel, I., Yolcu Oztel, G.: Human monkeypox classification from skin lesion images with deep pre-trained network using mobile application. *Journal of Medical Systems* **46**(11), 79 (2022)
- [7] Badiuzzaman, I.S.M.: Assessment of the usage of mobile applications (apps) in medical imaging among medical imaging students. *International Journal of Allied Health Sciences* **2**(2), 347–356 (2018)
- [8] Aldamani, R., Abuhani, D.A., Shanableh, T.: Lungvision: X-ray imagery classification for on-edge diagnosis applications. *Algorithms* **17**(7), 280 (2024)
- [9] Sato, S.: Plasmodium—a brief introduction to the parasites causing human malaria and their basic biology. *Journal of physiological anthropology* **40**(1), 1 (2021)
- [10] Mmileng, O.P., Whata, A., Olusanya, M., Mhlango, S.: Application of convnext with transfer learning and data augmentation for malaria parasite detection in resource-limited settings using microscopic images. *medRxiv*, 2024–10 (2024)
- [11] Google: About Android App Bundles — Android Developers — developer.android.com. [https://developer.android.com/guide/app-bundle#size\\_restrictions](https://developer.android.com/guide/app-bundle#size_restrictions). [Accessed 08-04-2024] (2024)
- [12] Apple: Maximum build file sizes - Reference - App Store Connect - Help - Apple Developer — developer.apple.com. <https://developer.apple.com/help/app-store-connect/reference/maximum-build-file-sizes/>. [Accessed 08-04-2024] (2024)
- [13] Gholami, A., Kim, S., Dong, Z., Yao, Z., Mahoney, M.W., Keutzer, K.: A survey of quantization methods for efficient neural network inference. In: *Low-Power Computer Vision*, pp. 291–326. Chapman and Hall/CRC, ??? (2022)
- [14] Organization, W.H.: Malaria. Accessed: 2024-12-03 (2024). <https://www.who.int/news-room/fact-sheets/detail/malaria>
- [15] Kang, J.-M., Cho, P.-Y., Moe, M., Lee, J., Jun, H., Lee, H.-W., Ahn, S.K., Kim, T.I., Pak, J.H., Myint, M.K., *et al.*: Comparison of the diagnostic performance of microscopic examination with

- nested polymerase chain reaction for optimum malaria diagnosis in upper myanmar. *Malaria journal* **16**, 1–8 (2017)
- [16] Wilson, M.L.: Laboratory diagnosis of malaria: conventional and rapid diagnostic methods. *Archives of Pathology and Laboratory Medicine* **137**(6), 805–811 (2013)
  - [17] Jutel, A., Lupton, D.: Digitizing diagnosis: a review of mobile applications in the diagnostic process. *Diagnosis* **2**(2), 89–96 (2015)
  - [18] Contreras-Naranjo, J.C., Wei, Q., Ozcan, A.: Mobile phone-based microscopy, sensing, and diagnostics. *IEEE Journal of Selected Topics in Quantum Electronics* **22**(3), 1–14 (2015)
  - [19] Wood, C.S., Thomas, M.R., Budd, J., Mashamba-Thompson, T.P., Herbst, K., Pillay, D., Peeling, R.W., Johnson, A.M., McKendry, R.A., Stevens, M.M.: Taking connected mobile-health diagnostics of infectious diseases to the field. *Nature* **566**(7745), 467–474 (2019)
  - [20] Narrillos-Moraza, Á., Gómez-Martínez-Sagrera, P., Amor-García, M.Á., Escudero-Vilaplana, V., Collado-Borrell, R., Villanueva-Bueno, C., Gómez-Centurió, I., Herranz-Alonso, A., Sanjurjo-Sáez, M.: Mobile apps for hematological conditions: review and content analysis using the mobile app rating scale. *JMIR mHealth and uHealth* **10**(2), 32826 (2022)
  - [21] Jain, S., Naicker, D., Raj, R., Patel, V., Hu, Y.-C., Srinivasan, K., Jen, C.-P.: Computational intelligence in cancer diagnostics: a contemporary review of smart phone apps, current problems, and future research potentials. *Diagnostics* **13**(9), 1563 (2023)
  - [22] Rokh, B., Azarpeyvand, A., Khanteymoori, A.: A comprehensive survey on model quantization for deep neural networks in image classification. *ACM Transactions on Intelligent Systems and Technology* **14**(6), 1–50 (2023)
  - [23] Elthakeb, A.T., Pilligundla, P., Miresghallah, F., Cloninger, A., Esmailzadeh, H.: Divide and conquer: Leveraging intermediate feature representations for quantized training of neural networks. In: *International Conference on Machine Learning*, pp. 2880–2891 (2020). PMLR
  - [24] Park, E., Yoo, S., Vajda, P.: Value-aware quantization for training and inference of neural networks. In: *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 580–595 (2018)
  - [25] Yang, F., Poostchi, M., Yu, H., Zhou, Z., Silamut, K., Yu, J., Maude, R.J., Jaeger, S., Antani, S.: Deep learning for smartphone-based malaria parasite detection in thick blood smears. *IEEE journal of biomedical and health informatics* **24**(5), 1427–1438 (2019)
  - [26] Loddo, A., Di Ruberto, C., Kocher, M., Prod’Hom, G.: Mp-idb: the malaria parasite image database for image processing and analysis. In: *Processing and Analysis of Biomedical Information: First International SIPAIM Workshop, SaMBa 2018, Held in Conjunction with MICCAI 2018, Granada, Spain, September 20, 2018, Revised Selected Papers 1*, pp. 57–65 (2019). Springer
  - [27] Chaudhry, H.A.H., Farid, M.S., Fiandrotti, A., Grangetto, M.: A lightweight deep learning architecture for malaria parasite-type classification and life cycle stage detection. *Neural Computing and Applications* **36**(31), 19795–19805 (2024)
  - [28] Arshad, Q.A., Ali, M., Hassan, S.-u., Chen, C., Imran, A., Rasul, G., Sultani, W.: A dataset and benchmark for malaria life-cycle classification in thin blood smear images. *Neural Computing and Applications* **34**(6), 4473–4485 (2022)
  - [29] Abbas, S.S., Dijkstra, T.M.H.: Malaria-Detection-2019. <https://doi.org/10.17632/5bf2kmwvfn.1> . <https://data.mendeley.com/datasets/5bf2kmwvfn/1>
  - [30] Lacuna Malaria Detection Challenge. Zindi Africa. Accessed: 2024-12-03 (2021). <https://zindi.africa/competitions/lacuna-malaria-detection-challenge>



- [31] Breesha, R., Alagu, S.: Malarial parasite detection based on smartphone microscopic imaging using deep learning approach. In: Proceedings of 2nd International Conference on Artificial Intelligence: Advances and Applications: ICAIAA 2021, pp. 565–577 (2022). Springer
- [32] Yang, F., Yu, H., Silamut, K., Maude, R.J., Jaeger, S., Antani, S.: Parasite detection in thick blood smears based on customized faster-rcnn on smartphones. In: 2019 IEEE Applied Imagery Pattern Recognition Workshop (AIPR), pp. 1–4 (2019). IEEE
- [33] Maturana, C.R., Oliveira, A.D., Nadal, S., Serrat, F.Z., Sulleiro, E., Ruiz, E., Bilalli, B., Veiga, A., Espasa, M., Abelló, A., *et al.*: imaging: a novel automated system for malaria diagnosis by using artificial intelligence tools and a universal low-cost robotized microscope. *Frontiers in microbiology* **14**, 1240936 (2023)
- [34] Nakasi, R., Mwebaze, E., Zawedde, A.: Mobile-aware deep learning algorithms for malaria parasites and white blood cells localization in thick blood smears. *Algorithms* **14**(1), 17 (2021)
- [35] Liu, R., Liu, T., Dan, T., Yang, S., Li, Y., Luo, B., Zhuang, Y., Fan, X., Zhang, X., Cai, H., *et al.*: Aidman: An ai-based object detection system for malaria diagnosis from smartphone thin-blood-smear images. *Patterns* **4**(9) (2023)
- [36] Loh, D.R., Yong, W.X., Yapeter, J., Subburaj, K., Chandramohanadas, R.: A deep learning approach to the screening of malaria infection: Automated and rapid cell counting, object detection and instance segmentation using mask r-cnn. *Computerized Medical Imaging and Graphics* **88**, 101845 (2021)
- [37] Özbilge, E., Giler, E., Ozbilge, E.: Ensembling object detection models for robust and reliable malaria parasite detection in thin blood smear microscopic images. *IEEE Access* (2024)
- [38] Nakasi, R., Mwebaze, E., Zawedde, A., Tusubira, J., Akera, B., Maiga, G.: A new approach for microscopic diagnosis of malaria parasites in thick blood smears using pre-trained deep learning models. *SN Applied Sciences* **2**, 1–7 (2020)
- [39] Murmu, A., Kumar, P.: Dlrnet: deep learning with random forest network for classification and detection of malaria parasite in blood smear. *Multimedia Tools and Applications*, 1–23 (2024)
- [40] Yu, H., Yang, F., Rajaraman, S., Ersoy, I., Moallem, G., Poostchi, M., Palaniappan, K., Antani, S., Maude, R.J., Jaeger, S.: Malaria screener: a smartphone application for automated malaria screening. *BMC Infectious Diseases* **20**, 1–8 (2020)
- [41] Li, S., Du, Z., Meng, X., Zhang, Y.: Multi-stage malaria parasite recognition by deep learning. *GigaScience* **10**(6), 040 (2021)
- [42] Jaganathan, S.C.B., Jaisingh, W., Dhanushraj, M., Pari, S., Sah, R., Dhananchezhian, S.: Convolutional neural network detection and classification of blood cell images from thin blood smear for the presence of malarial parasite. In: 2023 International Conference on New Frontiers in Communication, Automation, Management and Security (ICCAMS), vol. 1, pp. 1–10 (2023). IEEE
- [43] Islam, M.S.B., Islam, J., Islam, M.S., Sumon, M.S.I., Nahiduzzaman, M., Murugappan, M., Hasan, A., Chowdhury, M.E.: Development of low cost, automated digital microscopes allowing rapid whole slide imaging for detecting malaria. In: Surveillance, Prevention, and Control of Infectious Diseases: An AI Perspective, pp. 73–96. Springer, ??? (2024)
- [44] Shewajo, F.A., Fante, K.A.: Tile-based microscopic image processing for malaria screening using a deep learning approach. *BMC Medical Imaging* **23**(1), 39 (2023)
- [45] Nugroho, H.A., Nurfauzi, R.: Deep learning approach for malaria parasite detection in thick blood smear images. In: 2021 17th International Conference on Quality in Research (QIR): International Symposium on Electrical and Computer Engineering, pp. 114–118 (2021). IEEE

- [46] Rameen, I., Shahadat, A., Mehreen, M., Razzaq, S., Asghar, M.A., Khan, M.J.: Leveraging supervised machine learning techniques for identification of malaria cells using blood smears. In: 2021 International Conference on Digital Futures and Transformative Technologies (ICoDT2), pp. 1–6 (2021). IEEE
- [47] Ambarka, A., Djara, T., Sobabe, A.-A., Fayomi, H.: Prediction of malaria plasmodium stage and type through object detection. In: 2023 5th International Conference on Bio-engineering for Smart Technologies (BioSMART), pp. 1–4 (2023). IEEE
- [48] Zedda, L., Loddo, A., Di Ruberto, C.: A deep architecture based on attention mechanisms for effective end-to-end detection of early and mature malaria parasites. *Biomedical Signal Processing and Control* **94**, 106289 (2024)
- [49] Yu, H., Mohammed, F.O., Abdel Hamid, M., Yang, F., Kassim, Y.M., Mohamed, A.O., Maude, R.J., Ding, X.C., Owusu, E.D., Yerlikaya, S., *et al.*: Patient-level performance evaluation of a smartphone-based malaria diagnostic application. *Malaria Journal* **22**(1), 33 (2023)
- [50] Weiss, K., Khoshgoftaar, T.M., Wang, D.: A survey of transfer learning. *Journal of Big data* **3**, 1–40 (2016)
- [51] Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., Fei-Fei, L.: Imagenet: A large-scale hierarchical image database. In: 2009 IEEE Conference on Computer Vision and Pattern Recognition, pp. 248–255 (2009). Ieee
- [52] He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 770–778 (2016)
- [53] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., Chen, L.-C.: Mobilenetv2: Inverted residuals and linear bottlenecks. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 4510–4520 (2018)
- [54] Huang, G., Liu, Z., Van Der Maaten, L., Weinberger, K.Q.: Densely connected convolutional networks. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 4700–4708 (2017)
- [55] Zoph, B., Vasudevan, V., Shlens, J., Le, Q.V.: Learning transferable architectures for scalable image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 8697–8710 (2018)
- [56] Tan, M., Le, Q.: Efficientnet: Rethinking model scaling for convolutional neural networks. In: International Conference on Machine Learning, pp. 6105–6114 (2019). PMLR
- [57] Khanam, R., Hussain, M.: Yolov11: An overview of the key architectural enhancements. arXiv preprint arXiv:2410.17725 (2024)
- [58] Developers, A.: App Bundle Size Restrictions. <https://developer.android.com/guide/app-bundle#size-restrictions>. Accessed: 2024-12-04 (2024)
- [59] Developer, A.: Maximum Build File Sizes. <https://developer.apple.com/help/app-store-connect/reference/maximum-build-file-sizes/>. Accessed: 2024-12-05 (2024)